정규교과 3/18 · 오후

FastAPI–PostgreSQL 연동
지식 관리 API 완성

RAG·AI Agent 플랫폼의 데이터 기반
Day 19 · 연결 → ORM CRUD → 오류·Migration → 통합 검증

2026.11.12 (목) | 13:30~18:20 | RAG·AI Agent 플랫폼 기초

한로 강민준

현업 사례 | AI 도우미가 쓰는 지식은 누가 관리할까?

“신입 개발자에게 안내할 가상환경 정책이 달라졌습니다. AI 도우미는 무엇을 기준으로 답해야 할까요?”

지식 담당자의 업무 · 오늘

관리 화면·/docs에서 안내를 등록·수정
FastAPI → ORM·Session → PostgreSQL
원본의 저장과 변경을 검증

→

AI 도우미의 업무 · 이후

직원 질문을 분석하고 필요한 자료를 찾음
RAG 검색·LLM 생성·Agent 도구 실행
최신 근거와 접근 권한을 별도 확인

오늘 만드는 것: 원본 관리

이후: 검색용 인덱스 반영

이후: Agent 답변·도구

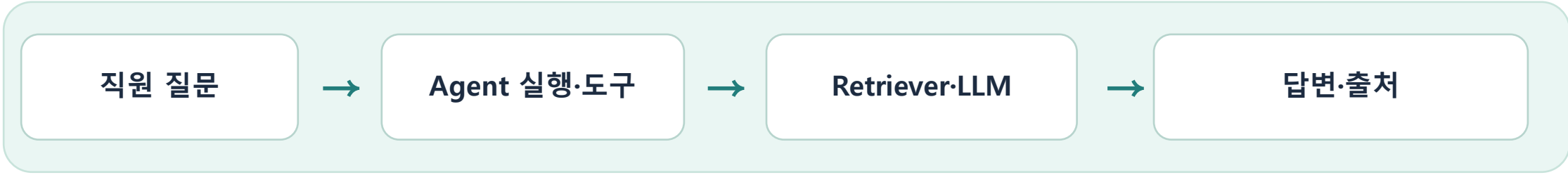
현업 질문: 답변용 지식이 바뀌었다면, 원본을 안전하게 등록·수정하고 반영 여부를 확인해야 한다.

AI Agent 전체 구조에서 오늘의 작업 위치

지식을 관리하는 경로 · 오늘 구현



질문을 처리하는 경로 · 향후 구성 예시



원본 지식 → (별도 동기화·인덱싱) → 검색 자료 · API가 존재한다고 Agent 도구가 자동 등록되지는 않음

PostgreSQL은 원본 지식 저장 계층이다. 모델·Retriever·Agent와 같은 역할이 아니다.

AI 플랫폼에서 DB 필드 하나까지 점점 확대한다

1 AI Agent 서비스 — 사람의 작업 요청을 안전한 API·도구에 연결

2 지식 관리 서비스 — Agent가 참고할 원본 지식의 생명주기 관리

3 FastAPI 엔드포인트 — POST /api/knowledge : 요청·응답 계약

4 ORM·Session — Python 객체를 DB 행으로 처리 · 확정·취소

5 테이블·행·필드 — knowledge.category_id → categories.id

큰 개념에서 작은 개념으로 이동해도 같은 업무 요청을 추적한다.

오전의 독립 실습이 오후에는 한 요청으로 연결된다

오전 · 각각 동작을 확인

FastAPI → JSON 파일 CRUD
pgAdmin → PostgreSQL SQL CRUD
두 경로는 별개였음
API를 호출해도 DB는 자동 변경되지 않음

→

오후 · 하나의 서비스로 연결

POST /api/knowledge
→ FastAPI 입력 검증
→ ORM·Session → PostgreSQL
→ 201 응답 + SQL에서 같은 ID 재조회

증거: API 응답 id = SQL에서 조회한 knowledge.id = 다시 GET한 id

API 성공 응답만으로 끝나지 않는다. 같은 ID의 실제 DB 행까지 확인해야 한다.

FastAPI 안에서는 Route → 함수 → 매개변수로 들어간다

① HTTP 요청

POST /api/knowledge
Body: title·content·category
/docs 또는 다른 클라이언트

→

② FastAPI Route

@app.post(...)
KnowledgeWrite 입력 검증
실제 처리 함수 연결

→

③ 처리 함수 안

payload: KnowledgeWrite
db: DbSession
분류 확인·ORM 저장 호출

```
def create_knowledge(payload: KnowledgeWrite, db: DbSession):
```

Agent 도구로 활용하려면 Method·Path·입력 스키마·권한을 계약으로 다뤄야 한다.

ORM 클래스에서 테이블·행·외래 키까지 확대한다

요청 JSON · 업무 용어

category = "learning"

→

ORM · DB 용어

Knowledge.category_id = 1

public.categories · 분류 테이블

id = 1 name = "learning"

→

public.knowledge · 지식 테이블

id = 실제 발급값 category_id = 1

PK(행 식별) · FK(참조 검사) · JOIN(분류명 조회)은 서로 다른 책임이다.

외부 API는 category 문자열, 내부 DB는 category_id 외래 키를 쓴다.

Session은 요청 하나의 DB 작업 범위를 관리한다

Engine / Pool: 애플리케이션에서 재사용하는 DB 연결 기반

요청 A · POST 새 지식

Session A · 읽기·추가·확정 또는 취소

서로 공유하지 않는 작업 상태

요청 B · GET 목록

Session B · 조회·정리

서로 공유하지 않는 작업 상태

Psycopg Driver → PostgreSQL

Session: ORM 객체·SQL·트랜잭션의 작업 공간 | 요청이 끝나면 Session 자원을 정리

Engine/Pool은 재사용하고 Session은 요청 단위로 생성·정리한다. close는 commit이 아니다.

flush 이후에도 COMMIT 전이면 아직 미확정이다

1 분류 확인·추가

categories

→

2 지식 객체 추가

knowledge

→

3 flush()

ID 확인·미확정

같은 Session · 같은 트랜잭션

정상 → COMMIT

새 분류와 지식 함께 확정

오류 → ROLLBACK

현재 미확정 변경 함께 취소

API의 201 응답은 COMMIT 성공 이후 반환 · DB 제약 오류와 API 입력 422는 다른 단계

분류와 지식은 하나의 작업으로 성공하거나 취소해야 한다. flush는 commit의 동의어가 아니다.

운영 개발은 스키마 변경과 API 검증을 나누어 본다

구조 변경 · Alembic

ORM Model에 source_label 같은 열 추가
→ Revision 초안·DDL 검토
→ 연습 DB에 upgrade
→ 실제 열·기존 행·이력 확인

동작 검증 · API + DB

POST 201 + 응답 id
→ SQL에서 같은 id·본문·분류 조회
→ PUT 200 / DELETE 204 확인
→ 실패 요청·재시작 후 상태 확인

주 프로젝트 DB와 Migration·테스트 DB는 분리

ORM 코드를 수정해도 실제 DB 열은 자동 변경되지 않는다. 변경과 검증의 증거를 남긴다.

오늘의 일일프로젝트는 Agent의 원본 지식 계층 완성

A · 끝까지 유지

POST → API 201
SQL 같은 ID 확인
재시작 뒤 보존

B · 본문 수정

PUT → API 200
DB 같은 ID·변경값 확인
기존 행·분류 보존

C · 삭제

DELETE → 빈 204
DB 조회 결과 0행
GET은 404

API 다섯 경로 + Session·트랜잭션 + 같은 DB 행 검증

별도 연습 DB에서 Migration 한 건 적용 | 기본필수는 /docs·pgAdmin 수동 검증으로 완료

지금 완성: 원본 지식 관리

다음 단계: RAG·Agent 연결

오늘: API → PostgreSQL까지 구현·검증 | 이후: RAG 인덱싱·Agent 도구·권한 정책 연결

오전에는 각각, 오후에는 하나로 연결한다

오전 · 각각 확인

FastAPI → JSON 파일
SQL → PostgreSQL

API를 호출해도 DB가
자동으로 바뀌지는 않았다.

오후 · 같은 DB에 반영

FastAPI → ORM·Session → DB
SQL로 같은 ID를 다시 조회

API의 새 지식과
DB의 새 행이 일치해야 한다.

시연: POST 한 번 → 응답 id 확인 → pgAdmin에서 같은 id 조회 → API 재조회

AI 플랫폼에서 오늘 만드는 부분

지식 관리 기능을 먼저 연결한 뒤, 이후 검색·생성·도구 실행을 붙인다.

1
클라이언트
오늘은 /docs

→

2
FastAPI
입력·응답 약속

→

3
ORM·Session
조회·변경·확정

→

4
PostgreSQL
지식·분류·제약

RAG·Agent 처리 코드는 후속 수업이다. API·DB 연동만으로 AI 답변이 생성되지는 않는다.

참고 S01 · S03 | 공식 링크: 발표자 노트·참고자료

오후 5교시 · 먼저 연결하고, 마지막에 완성한다

시간	핵심 내용	설명·시연	실습·확인
13:30~14:20	설정·ORM 흐름·첫 DB 조회	25분	25분
14:30~15:20	Schema·분류 연결·POST·PUT·DELETE	25분	25분
15:30~16:20	트랜잭션·오류·데이터 보존	20분	30분
16:30~17:20	Migration·API/DB 통합 검증	20분	30분
17:30~18:20	작은 일일프로젝트·시연	10분	40분

수업 250분 = 설명·시연 100분 + 실습·확인 150분 | 쉬는 시간 10분씩 별도

세 단계 모두 DB 연동 CRUD까지 완성한다

기본필수
핵심 개념·동작

제공 설정·Model·저장 골격에서 핵심 호출을 완성한다.
/docs와 SQL로 요청·응답·DB 행을 직접 확인한다.

응용
작은 기능 확장

조회 조건 하나와 직접 작성할 로직을 조금 늘린다.
반복 코드를 함수·간단한 파일로 분리한다.

심화
역할 분리·검증

Router·Schema·Service·Repository의 책임을 나눈다.
제공 테스트 틀에서 오류·DB 반영·격리를 검증한다.

수준 하나를 선택한다. 기본필수만으로 공통 평가 100점이 가능하다.

어느 DB를 연결할지 먼저 고정한다

대상	사용 방법	보호할 것
오전 CRUD DB	기본 day19_crud_b / 응용 _a / 심화 _h	공통 두 테이블·기존 행 보존
다른 정상 DB	knowledge_lab 등: 구조 확인 후 명시적으로 선택	DB명·실제 초기 개수 기록
구축 연습 DB	day19_build_*는 비었거나 미완성일 수 있음	주 연결 대상으로 자동 선택 금지
별도 연습 DB	day19_migration_lab / day19_api_test	Migration·자동 테스트만 사용

기존 DB를 DROP·재초기화하지 않는다. API 시작 시 create_all·Seed 자동 실행을 넣지 않는다.

바꾸지 않을 약속 · 경로·입력·응답

요청	입력	정상 결과
GET /api/knowledge	없음 · 응용부터 category 조건	200 · id 오름차순 배열
GET /api/knowledge/{knowledge_id}	정수 Path ID	200 · 지식 하나
POST /api/knowledge	title · content · category	201 · DB가 발급한 id 포함
PUT /api/knowledge/{knowledge_id}	Path ID + 세 필드 모두	200 · 같은 id의 수정 결과
DELETE /api/knowledge/{knowledge_id}	정수 Path ID	204 · 본문 없음

없는 지식: 404 {"detail":"Knowledge not found"} | 잘못된 입력: 기본 422

참고 S07 · S08 · S09 | 공식 링크: 발표자 노트·참고자료

기존 환경에 필요한 패키지만 추가한다

```
python -m pip show fastapi pydantic uvicorn
python -m pip install "sqlalchemy>=2.0,<2.1"
python -m pip install "psycopg[binary]>=3,<4"
python -m pip install pydantic-settings alembic
# 심화 자동 테스트만: httpx pytest
python -m pip show sqlalchemy psycopg alembic

# 오후 앱의 프로젝트 최상위에서 실행
python -m uvicorn main:app --port 8001 --reload
```

Windows · VS Code

정상 가상환경을 재사용한다.

오전 서버는 종료한다.
오후는 8001로 구분한다.

새 패키지 설치·권한 문제는
수업 전 확인하는 것이 좋다.

오후 주소
127.0.0.1:8001/docs

설치 명령은 예시다. 실제 검증 조합은 부록에 구분한다. GPU·CUDA는 필요 없다.

.env · 접속 정보와 코드를 분리한다

```
# .env — 실제 값은 제출·Git에 넣지 않는다.
DB_HOST=localhost
DB_PORT=5432
DB_NAME=day19_crud_b
DB_USER=내_수업용_계정
DB_PASSWORD='내_비밀번호'

# .gitignore
.env
.env.*
!.env.example
.venv/
__pycache__/
```

필요한 값

pgAdmin에서 쓰던 서버·포트·
DB·계정을 정확히 맞춘다.

.env는 암호화 저장소가 아니다.
.env.example에는 빈 예시만 둔다.

환경변수도 같은 이름인지 확인.

DB 이름이 틀리면
다른 데이터를 보게 된다.

URL.create · 비밀번호를 문자열로 조립하지 않는다

```
# db.py 일부 · settings는 부록 A02
url = URL.create(
    "postgresql+psycopg",
    username=settings.db_user,
    password=settings.db_password.get_secret_value(),
    host=settings.db_host,
    port=settings.db_port,
    database=settings.db_name,
)
engine = create_engine(
    url, pool_pre_ping=True,
    hide_parameters=True,
    connect_args={"connect_timeout": 5},
)
```

역할을 구분한다

URL.create: 구성요소를 전달
@·%가 있는 비밀번호도 처리

Engine: 연결을 빌리는 기반
생성만으로 접속 성공은 아님

pool_pre_ping은 생존 점검이다.

Engine은 앱에서 재사용
Session은 요청마다 생성

Engine·Driver·ORM·Session을 나누어 읽는다

같은 기능의 다른 이름이 아니다. 오늘은 동기 def와 일반 Session을 사용한다.

Engine / Pool

연결 생성·재사용

Psycopg

PostgreSQL 통신

ORM Model

클래스 ⇄ 테이블

Session

객체·SQL·트랜잭션

역할 비교 그림이다. 요청의 실행 순서는 앞의 연결 흐름에서 확인한다.

참고 S02 · S03 · S04 · S06 · S18 | 공식 링크: 발표자 노트·참고자료

같은 DB를 보고 있는지 실제로 확인한다

```
# check_database.py · 로컬 터미널에서만 확인
from sqlalchemy import text
from db import engine

with engine.connect() as conn:
    info = conn.execute(text(
        "SELECT current_database(), current_user"
    )).one()
    count = conn.execute(text(
        "SELECT count(*) FROM public.knowledge"
    )).scalar_one()
print("DB:", info[0], "USER:", info[1])
print("knowledge count:", count)
```

확인의 순서

DB·사용자 확인
→ public.knowledge 읽기
→ pgAdmin 결과와 대조

행 수를 3으로 고정하지 않는다.
오전 변경이 남아 있을 수 있다.

접속만 성공 ≠ 구조 일치
실제 테이블도 확인

같은 지식 · 외부 표현과 내부 표현은 다르다

API · 프로그램과의 약속

id · title · content · category

category는 "learning" 같은 문자열

POST와 PUT의 세 입력 필드는
오전과 동일하다.

DB · 관계와 규칙

knowledge: id / title / content / category_id

categories: id / name

category_id로 연결하고
응답에서는 분류명을 돌려준다.

입력 모델은 테이블이 아니며, ORM 선언만으로 기존 테이블이 변경되지 않는다.

요청마다 Session을 만들고 끝나면 정리한다

```
# db.py · Engine은 앞 코드에서 생성
SessionLocal = sessionmaker(
    bind=engine, autoflush=False,
    expire_on_commit=False,
)
def get_db():
    with SessionLocal() as db:
        yield db

# main.py
DbSession = Annotated[Session, Depends(get_db)]
```

yield 앞·뒤

yield 앞: 요청의 Session 준비
yield 값: 함수 매개변수로 전달
끝나면: Session.close

close가 성공 저장은 아니다.
쓰기 확정은 commit이 담당한다.

전역 공유 Session 금지
요청별로 생성·정리

첫 GET · SQL로 넣은 지식이 API에 보인다

```
# main.py · Model·Schema·to_read는 제공 골격
@app.get("/api/knowledge",
         response_model=list[KnowledgeRead])
def list_knowledge(db: DbSession):
    stmt = select(Knowledge, Category.name).join(
        Category,
        Knowledge.category_id == Category.id,
    ).order_by(Knowledge.id)
    rows = db.execute(stmt).all()
    return [to_read(row, name) for row, name in rows]
```

SQL과 대응한다

select: 지식과 분류명 선택
join: FK와 PK를 연결
order_by: id 오름차순
execute: 실제 조회

to_read: 응답 네 필드로 변환

200 · DB의 실제 목록
빈 목록은 200 · []

실습 01 · 설정부터 실제 DB 목록 조회까지

기본필수 핵심 개념·동작	제공 설정·Model 골격으로 접속 대상을 맞춘다. API 목록과 pgAdmin의 지식 ID·제목·분류명을 대조한다.
응용 작은 기능 확장	목록 조회 함수를 연결하고 category 조건 하나를 추가한다. 일치 결과와 200·[]를 직접 확인한다.
심화 역할 분리·검증	설정·DB 수명·조회 역할을 분리한다. 테스트 연결을 바꿀 수 있는 지점을 설명한다.

완료: /docs가 아니라 실제 DB의 지식이 API 응답으로 보인다.

ORM Model과 Pydantic은 서로 다른 규칙을 읽는다

ORM Model · DB 쪽

Category·Knowledge 클래스
열·자료형·PK·FK·CHECK

Session이 객체를 추적한다.
DB의 실제 구조와 맞아야 한다.

Pydantic · API 쪽

KnowledgeWrite·KnowledgeRead
JSON의 필수 값·길이·응답 필드

category는 외부 문자열이다.
DB의 내부 열을 그대로 공개하지 않는다.

from_attributes=True 하나로 category_id가 category 문자열로 자동 변환되지는 않는다.

참고 S03 · S08 | 공식 링크: 발표자 노트·참고자료

Category · 분류 테이블과 맞춘다

```
# models.py · Base와 import는 부록 A04
class Category(Base):
    __tablename__ = "categories"
    __table_args__ = (
        CheckConstraint(
            "char_length(name) BETWEEN 1 AND 30",
            name="categories_name_length"),
        {"schema": "public"},
    )
    id: Mapped[int] = mapped_column(
        BigInteger, Identity(always=True),
        primary_key=True)
    name: Mapped[str] = mapped_column(
        String(30), nullable=False, unique=True)
```

어제 DDL과 대조

BigInteger ⇄ BIGINT
Identity(always=True) ⇄ ALWAYS
String(30) ⇄ VARCHAR(30)

분류명은 UNIQUE다.
기존 테이블을 다시 생성하지 않는다.

클래스 선언 ≠ CREATE 실행
DB 규칙은 실제 DB에도 있어야 함

Knowledge · 지식의 필드와 FK를 맞춘다

```
# models.py · 길이 CHECK는 부록 A05
class Knowledge(Base):
    __tablename__ = "knowledge"
    # __table_args__: CHECK 2개 + schema public
    id: Mapped[int] = mapped_column(
        BigInteger, Identity(always=True),
        primary_key=True)
    title: Mapped[str] = mapped_column(
        String(80), nullable=False)
    content: Mapped[str] = mapped_column(
        Text, nullable=False)
    category_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("public.categories.id",
            name="knowledge_category_fk",
            ondelete="RESTRICT"), nullable=False)
```

이번 데이터 규칙

제목: 1~80자
본문: 1~2,000자
분류: 필수 BIGINT FK

지식 제목은 중복 허용이다.
분류 삭제는 FK가 보호한다.

전체 정의는 부록 A05
같은 클래스를 두 번 선언하지 않음

입력 세 필드와 응답 네 필드를 유지한다

```python
# main.py · 응용부터 schemas.py로 분리 가능
class KnowledgeWrite(BaseModel):
    title: str = Field(min_length=1, max_length=80)
    content: str = Field(min_length=1, max_length=2000)
    category: str = Field(min_length=1, max_length=30)

class KnowledgeRead(KnowledgeWrite):
    id: int

def to_read(row: Knowledge, category: str):
    return KnowledgeRead(
        id=row.id, title=row.title,
        content=row.content, category=category,
    )
```

계약은 그대로

POST·PUT: 세 입력 모두 필수
id: DB가 발급한 값

category_id → 분류명으로 변환

PUT에 content만 보내면
기본 422다.

id · title · content · category
이 네 필드만 응답

분류명은 찾고, 없으면 같은 작업 안에서 만든다

```
# main.py · transaction 안에서만 호출
def get_or_create_category(db, name):
    category = db.scalar(
        select(Category).where(Category.name == name)
    )
    if category is None:
        category = Category(name=name)
        db.add(category)
        db.flush()  # category.id 확보, 아직 미확정
    return category
```

명시한 수업 정책

이미 있는 분류 → 그 ID 재사용
없는 분류 → 새 분류 생성

여기서 commit하지 않는다.
분류와 지식을 함께 확정한다.

도우미는 제공 골격이다.

외부 입력: category 문자열
내부 저장: category_id

동시 생성 충돌은 부록의 오류 정책으로 처리한다. "조회 후 생성"이 동시성 전체를 해결하지는 않는다.

참고 S02 · S10 | 공식 링크: 발표자 노트·참고자료

POST · 분류와 지식을 저장한 뒤 201을 반환한다

```
@app.post("/api/knowledge", status_code=201,
          response_model=KnowledgeRead)
def create_knowledge(payload: KnowledgeWrite,
                     db: DbSession):
    with transaction(db):
        category = get_or_create_category(
            db, payload.category)
        row = Knowledge(
            title=payload.title, content=payload.content,
            category_id=category.id)
        db.add(row)
        db.flush()
        result = to_read(row, category.name)
    return result
```

순서대로 추적

입력 검증 → 분류 ID 결정
→ 지식 객체 등록
→ flush로 DB 발급 id 확인
→ commit 성공 후 응답

transaction은 다음 교시에서 확대한다.

201의 id = DB 행의 id
next_id를 직접 관리하지 않음

PK로 찾고, 없으면 기존 404 계약을 지킨다

```
def knowledge_or_404(db, knowledge_id):
    row = db.get(Knowledge, knowledge_id)
    if row is None:
        raise HTTPException(404, "Knowledge not found")
    return row

@app.get("/api/knowledge/{knowledge_id}",
         response_model=KnowledgeRead)
def read_knowledge(knowledge_id: int, db: DbSession):
    row = knowledge_or_404(db, knowledge_id)
    category = db.get(Category, row.category_id)
    return to_read(row, category.name)
```

존재 확인과 변환

db.get: PK로 객체 하나
없으면 None

category_id로 부모를 찾고
이름을 응답에 넣는다.

정수 아님은 기본 422다.

없는 정수 ID → 404
목록 0건 → 200 · []

참고 S02 · S07 | 공식 링크: 발표자 노트·참고자료

PUT · ID를 유지하고 세 필드를 교체한다

```
@app.put("/api/knowledge/{knowledge_id}",
         response_model=KnowledgeRead)
def update_knowledge(knowledge_id: int,
                     payload: KnowledgeWrite,
                     db: DbSession):
    with transaction(db):
        row = knowledge_or_404(db, knowledge_id)
        category = get_or_create_category(
            db, payload.category)
        row.title = payload.title
        row.content = payload.content
        row.category_id = category.id
        db.flush()
        result = to_read(row, category.name)
    return result
```

조회부터 한다

없는 지식을 먼저 거부한다.
새 분류부터 만들지 않는다.

객체 속성을 바꾸면
flush/commit 때 UPDATE된다.

id와 지식 수는 유지한다.

같은 id · 바뀐 필드
없는 ID를 새로 생성하지 않음

DELETE · commit 후 본문 없는 204로 끝낸다

```
@app.delete("/api/knowledge/{knowledge_id}",
            status_code=204)
def delete_knowledge(knowledge_id: int,
                     db: DbSession):
    with transaction(db):
        row = knowledge_or_404(db, knowledge_id)
        db.delete(row)
    return Response(status_code=204)
```

삭제 범위를 확인

db.delete: 삭제 대상으로 표시
commit: 실제 삭제 확정

해당 지식만 삭제한다.
부모 분류는 자동 삭제하지 않는다
.

이후 상세 조회는 404다.

204의 본문 길이 0
{} · null · 메시지도 반환하지 않음

실습 02 · 다섯 API를 DB CRUD로 연결한다

기본필수
핵심 개념·동작

제공 도우미를 사용해 생성·PUT·삭제의 핵심 호출을 완성한다.
반환 ID를 SQL로 찾아 API와 DB의 값을 대조한다.

응용
작은 기능 확장

분류 필터와 함수 분리를 조금 더 직접 작성한다.
새 분류·기존 분류의 처리와 빈 조회 결과를 확인한다.

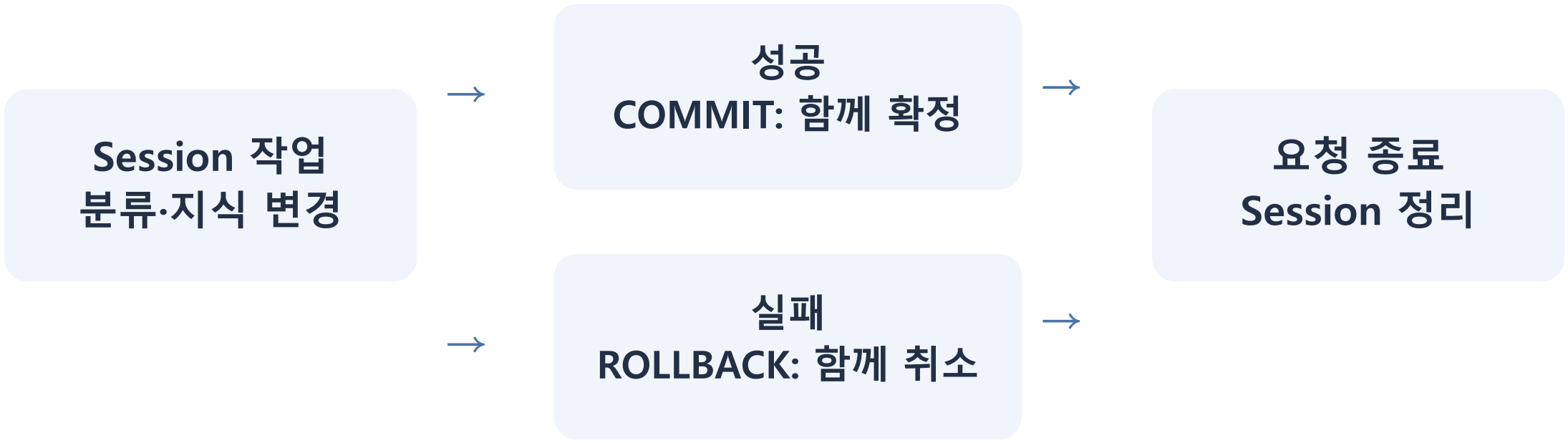
심화
역할 분리·검증

Router와 저장 로직을 분리하되 트랜잭션 경계는 하나로 둔다.
API 계약을 바꾸지 않고 같은 CRUD 결과를 확인한다.

완료: GET 목록·상세, POST, PUT, DELETE 모두 DB의 실제 상태와 일치한다.

성공과 실패는 서로 다른 종료 경로다

새 분류와 새 지식처럼 함께 맞아야 하는 변경을 하나의 작업으로 묶는다.



COMMIT 뒤 ROLLBACK을 실행한다고 이미 확정한 변경이 취소되는 것은 아니다.

참고 S02 · S19 | 공식 링크: 발표자 노트·참고자료

비슷해 보이는 메서드의 역할을 구분한다

메서드	하는 일	하지 않는 일
add(obj)	새 객체를 Session에 등록	호출 즉시 commit하지 않음
flush()	변경 SQL을 전송 · id 확보 가능	트랜잭션을 확정하지 않음
commit()	필요한 flush 후 변경 확정	모든 오류·경쟁을 자동 해결하지 않음
refresh(obj)	DB 값을 다시 객체에 읽음	저장 명령이 아님
rollback()	현재 미확정 변경 취소·실패 상태 정리	발급된 ID 번호까지 되돌리지 않음
close()	Session의 연결·상태 정리	저장을 자동 확정하지 않음

수업 코드: flush → 응답 객체 준비 → commit → return. refresh는 필요할 때만 사용한다.

참고 S02 · S11 | 공식 링크: 발표자 노트·참고자료

transaction · flush와 commit 실패를 모두 감싼다

```python
# db.py · 요청의 쓰기 경계를 하나로 정한다.
from contextlib import contextmanager

@contextmanager
def transaction(db):
    try:
        yield                # with 안의 조회·변경·flush
        db.commit()          # 정상일 때 한 번 확정
    except Exception:
        db.rollback()        # 오류가 나면 미확정 작업 취소
        raise                # 성공 응답으로 숨기지 않는다.
```

이 도우미의 전제

한 요청의 쓰기 전체에 사용
도우미 안에서 추가 commit 금지
중첩 transaction 호출 금지

HTTP 오류 변환은 별도다.
예외를 다시 전달한다.

commit 성공 전에는
201·200·204를 보내지 않음

새 분류만 남는 불완전한 저장을 막는다

잘못 나누면

새 분류 INSERT
→ 먼저 COMMIT
→ 지식 INSERT 실패

지식은 없고 새 분류만 남을 수 있다.

수업 코드의 경계

새 분류 INSERT·flush
→ 지식 INSERT·flush
→ 함께 COMMIT 또는 ROLLBACK

도우미의 flush는 중간 확정이 아니다.

422 요청은 저장 전 거부다. 실제 DB 실패의 rollback은 별도 시연·테스트로 확인한다.

참고 S02 · S10 · S19 | 공식 링크: 발표자 노트·참고자료

오류의 원인과 HTTP 응답을 함께 구분한다

상황	응답 정책	DB 확인
필수 값·길이·Path 형식 오류	422 · FastAPI 기본 검증	쓰기 코드로 들어가지 않음
정상 입력 + 없는 지식 ID	404 · Knowledge not found	새 지식·분류 생성 없음
알려진 분류 UNIQUE / FK 충돌	409 · Data conflict	현재 트랜잭션 rollback
확인된 연결 실패·접속 시간초과	503 · Database unavailable	성공으로 보고하지 않음
예상 밖 CHECK·SQL·설정 오류	500 · 일반 오류 메시지	내부 코드·구조 불일치 점검

409·503은 이 교안의 명시적 분류 정책이다. 모든 DB 오류를 "중복"으로 바꾸지 않는다.

참고 S07 · S16 · S17 · S20 | 공식 링크: 발표자 노트·참고자료

IntegrityError를 실제 제약 이름으로 분류한다

```
# main.py · 예외 처리기 일부, 전체는 부록 A09
code = getattr(exc.orig, "sqlstate", None)
diag = getattr(exc.orig, "diag", None)
name = getattr(diag, "constraint_name", None)
known = {
    ("23505", "categories_name_key"),
    ("23503", "knowledge_category_fk"),
}
if (code, name) in known:
    return JSONResponse(status_code=409,
        content={"detail": "Data conflict"})
return JSONResponse(status_code=500,
    content={"detail": "Database operation failed"})
```

원인을 읽는다

23505: UNIQUE 위반
23503: FK 위반

같은 예외 종류라도
실패한 제약은 다를 수 있다.

rollback은 쓰기 도우미가 담당.

지식 제목 중복은 허용
제목 중복 409를 만들지 않음

실패도 서로 다른 두 종류로 검증한다

검사	실행	해석
입력 검증	POST content="" → 422 기존 목록·행 내용 불변	쓰기 전 거부를 확인 DB rollback의 증거는 아님
DB 오류 시연	별도 DB의 Session에서 제약 위반 rollback 후 행·후속 조회 확인	같은 Session의 회복을 확인 시퀀스 번호는 원복 기준 아님
다음 HTTP 요청	오류 후 정상 POST → 201	다음 요청의 성공 확인 이전 Session 재사용 증거는 아님

기본필수: /docs·SQL 수동 대조 + 강사 시연 이해 | 심화: 별도 DB 테스트 보완

참고 S01 · S02 · S11 | 공식 링크: 발표자 노트·참고자료

응답이 이상하면 연결 대상부터 다시 본다

증상	먼저 확인	피할 행동
API 결과와 SQL 결과가 다름	서버 포트 · DB_NAME · schema · 실제 ID	Seed를 무작정 다시 넣기
DB를 못 찾거나 인증 실패	pgAdmin의 Host·Port·DB·계정과 비교	비밀번호·접속 문자열 공개
테이블·열 오류	ORM 매핑과 실제 DDL 대조	create_all로 모든 차이 해결 기대
PendingRollbackError	실패 후 rollback 여부 · 경계 중첩	같은 Session을 계속 사용
POST 재시도 후 중복 데이터	이전 요청 성공 여부·실제 DB 조회	실패처럼 보이면 무조건 재전송

실습 03 · 실패한 요청에서 지식이 보호되는가

기본필수
핵심 개념·동작

404·422 전후의 DB 지식과 분류를 비교한다.
제공된 rollback 시연에서 취소·정리 위치를 설명한다.

응용
작은 기능 확장

알려진 DB 충돌과 예상 밖 오류의 분류를 확인한다.
실패 후 정상 요청과 잘못된 분류 추가 여부를 점검한다.

심화
역할 분리·검증

별도 DB에서 flush·commit 실패 경로를 테스트한다.
같은 Session 복구와 새 요청 성공을 나누어 확인한다.

완료: 오류 응답·실제 데이터·트랜잭션 경계가 서로 맞는다.

Model 수정과 실제 DB 변경은 별도다

Python 코드가 바뀐 상태

ORM에 새 열을 선언했다.

기존 DB는 아직 옛 구조일 수 있다.

create_all은 일반적인
기존 열 변경·이력 관리 도구가 아니다.

DB 변경을 관리하는 방식

Revision에 변경 순서를 기록
→ 사람이 내용 검토
→ 대상 DB에 upgrade
→ 구조와 기존 데이터 확인

Alembic으로 이 과정을 관리한다.

오늘은 별도 연습 DB에서 작은 변경 한 건을 확인한다. 주 프로젝트 DB는 그대로 둔다.

참고 S12 · S13 | 공식 링크: 발표자 노트·참고자료

주 프로젝트·Migration·테스트 DB를 분리한다

사용 목적	시작 상태	금지·완료 기준
주 프로젝트 day19_crud_b 등	오전에 사용한 지식·분류 보존	초기 Revision·DROP·Seed 반복 금지
Migration 연습 day19_migration_lab	새 개인 빈 DB · 제공 초기 Revision	001 → 데이터 1건 → 002 → 데이터 보존
자동 테스트 day19_api_test	주 DB와 다른 전용 구조·가상 데이터	연결 대상 검사 후 테스트 · 생성 항목만 정리

DB명만 다른 URL인지가 아니라, 실제 Host·Port·Database와 접속 결과를 확인한다.

참고 S12 · S15 | 공식 링크: 발표자 노트·참고자료

작은 변경을 순서대로 적용하고 확인한다

연습 예: 지식의 출처 표기를 위한 source_label 열을 선택 항목으로 추가한다.

1 **001_initial**
두 테이블 생성

→

2 **기준 데이터**
가상 지식 1건

→

3 **002_source**
nullable 열 추가

→

4 **검증**
열·이력·기존 행

source_label 추가만으로 RAG 출처 검증이 구현되지는 않는다. API 네 필드 계약은 유지한다.

002_source · 추가와 되돌림을 함께 읽는다

```
# migration_demo/migrations/versions/002_source.py
from alembic import op
import sqlalchemy as sa
revision = "002_source"
down_revision = "001_initial"

def upgrade():
    op.add_column("knowledge",
        sa.Column("source_label", sa.String(120),
                  nullable=True), schema="public")

def downgrade():
    op.drop_column("knowledge", "source_label",
                   schema="public")
```

초안도 검토한다

revision: 현재 변경 ID
down_revision: 바로 앞 변경
upgrade: 새 열 추가
downgrade: 그 열 삭제

삭제하면 열 안의 데이터는 잃는다
.

NOT NULL 새 필드를
기존 행에 무작정 강제하지 않음

대상을 확인한 뒤 제공된 변경을 적용한다

```
# migration_demo 폴더에서 · .env.migration 사용
python -m alembic current
python -m alembic history
python -m alembic upgrade 001_initial
# 이 시점: 가상 지식 1건 추가 후 COMMIT
python -m alembic upgrade head
python -m alembic current

-- pgAdmin: day19_migration_lab에서 확인
SELECT column_name, is_nullable
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name = 'knowledge';
```

두 실행 위치

위: 터미널의 Alembic 명령
아래: pgAdmin의 SQL

주 프로젝트의 .env와 구분한다.
current의 head 표시와 함께
실제 열·기존 행도 확인한다.

002_source · source_label
기존 지식 값은 유지

autogenerate는 초안이고, stamp는 수리가 아니다

autogenerate

현재 DB ⇄ metadata 차이 비교

rename 의도가 drop/add로
나타나는 등 검토가 필요하다.

자동 생성 후 내용을 읽는다.

기존 DB 편입

기존 DDL과 초기 Revision 대조
→ 일치가 검증된 경우에만
이력 편입 절차를 별도 설계

stamp는 DDL을 실행하지 않는다.
실패했으니 stamp부터 하지 않는다.

기본필수는 제공 Revision 적용·확인에 집중한다. 기존 DB 이력 편입은 참고 설명이다.

참고 S12 · S13 | 공식 링크: 발표자 노트·참고자료

API 성공은 실제 DB 증거와 함께 확인한다

HTTP 확인	DB 확인	이것까지는 증명하지 않음
201과 생성된 id	같은 id의 지식·분류 관계 조회	모든 운영 장애에 대한 내구성
PUT 200	같은 id의 본문·분류 반영	동시 수정 충돌 전체 해결
DELETE 204 + 빈 본문	대상 0행 · 다른 행 보존	미래 벡터 인덱스 자동 삭제
422 / 404	기존 데이터 불변	같은 Session의 DB 오류 복구
재시작 후 GET	새 DB 연결의 값과 동일	DB 서버 장애·백업 복구 시험

기본필수: /docs + pgAdmin으로 충분하다. 심화만 제공 자동 테스트를 보완한다.

참고 S08 · S15 | 공식 링크: 발표자 노트·참고자료

응답의 id로 Test DB를 다시 조회한다

```
# 별도 DB로 get_db를 교체한 제공 fixture 사용
response = client.post("/api/knowledge", json={
    "title": "시험용 안내",
    "content": "공개 가능한 가상 문장입니다.",
    "category": "learning",
})
assert response.status_code == 201
created = response.json()
with TestSessionLocal() as db:
    row = db.get(Knowledge, created["id"])
    assert row is not None
    assert row.content == created["content"]
    category = db.get(Category, row.category_id)
    assert category.name == created["category"]
```

전제부터 확인

client가 전용 DB를 사용해야 한다.

같은 PK·본문·분류명 대조

테스트가 만든 행만 정리하고
마지막에 override도 해제한다.

상태코드 + 응답 + DB 행
세 가지를 같이 검사

실습 04 · 작은 Migration과 통합 검증

기본필수 핵심 개념·동작	별도 연습 DB에서 제공 Revision을 적용·확인한다. 주 API는 /docs·SQL로 CRUD 반영을 수동 대조한다.
응용 작은 기능 확장	변경 전후 열·기존 행을 비교하고 결과를 설명한다. 분류 필터·빈 목록·실패 입력 사례를 하나씩 추가한다.
심화 역할 분리·검증	제공 Test DB·override 골격에 확인문을 보완한다. 이력·실제 Schema·API·DB 행을 분리하여 검증한다.

완료: 적용 대상과 실제 변경을 설명한다. Migration 연습이 주 프로젝트 데이터를 바꾸면 안 된다.

오늘의 서비스는 이후 AI 기능의 기반이다

오늘 완성

지식 원본의 CRUD
API 입력·응답 약속
관계·제약·트랜잭션
오류와 DB 반영 검증

이후에 연결

문서·청크·출처 관리
검색 결과와 LLM 생성
Agent가 사용하는 읽기 도구
권한·승인·인덱스 갱신 정책

ORM 조회·category 필터는 의미 검색이 아니다. API Endpoint가 자동으로 Agent 도구가 되지는 않는다.

나만의 PostgreSQL 연동 AI 도움말 API

내가 정할 것

사용자와 주제 하나
가상의 지식 세 문장
A는 유지 · B는 수정 · C는 삭제

오전 내용을 재사용해도 좋다.

이미 배운 것을 연결

기존 API·ORM·DB 코드를 재사용
새 FastAPI 프로젝트를 다시 만들지 않음

API로 변경 → 같은 ID를 SQL로 확인
→ API 재조회·서버 재시작

LLM·벡터 검색·로그인 화면·추가 테이블은 만들지 않는다. 실제 API→DB 연결을 완성한다.

A 유지 · B 수정 · C 삭제를 끝까지 추적한다

단계	내가 할 일	통과 기준
시작	같은 DB·기존 행·시작 수 N 기록	기존 행은 보존
생성·조회	POST 3회 → 실제 A/B/C id 기록	API 201·200 / DB N+3
수정	B를 PUT한 뒤 SQL·GET 확인	B의 같은 id·수정 본문
실패	content="" POST / 정상 Body+없는 ID	422·404 / 기존 지식·분류 불변
삭제	C DELETE → SQL·GET 확인	204 빈 본문 / DB 0행 / GET404
재실행	API 재시작 + DB 새 연결로 확인	A·B·기존 행 보존 / 최종 N+2

ID는 실제 응답값을 쓴다. PUT은 세 필드 모두 보낸다. 실패 뒤 ID 번호의 연속성은 검사하지 않는다.

응답에서 받은 id를 SQL에 연결한다

```
# /docs의 POST Body 예시 · Python 코드 아님
{
  "title": "실습 환경 안내",
  "content": "실행 전에 가상환경을 확인합니다.",
  "category": "learning"
}

-- pgAdmin · 17은 실제 응답 id로 바꾼다.
SELECT k.id, k.title, k.content, c.name AS category
FROM public.knowledge AS k
JOIN public.categories AS c ON c.id = k.category_id
WHERE k.id = 17;
```

증거의 연결

POST 응답의 id를 읽는다.

같은 DB에서 SQL을 실행한다.
제목·본문·분류명을 대조한다.

다시 API GET으로 같은 id 조회.

오후에는 API와 SQL이
같은 DB 행을 읽는다.

마지막 50분 · 안내 6 + 수행 40 + 공유 4

경과	수행 내용	남길 결과
0~6분	목표·완료 기준·현재 코드 확인	DB 대상과 시작 상태
6~11분	주제·세 문장·A/B/C 역할 결정	세 필드 입력 준비
11~26분	미완성 CRUD 보완·생성·조회·B 수정	실제 ID와 SQL 반영
26~41분	오류·C 삭제·API 재시작·DB 새 조회	실패 보호·최종 N+2
41~46분	실행 방법·검증 근거 간단 정리	작은 README·실제 결과
46~50분	대표 시연·한 문장 회고	플랫폼에서 맡을 역할

시간은 운영 기준이다. 미완성자는 공통 CRUD부터, 완료자는 선택 확장 하나만 보완한다.

같은 연동 서비스를 수준에 맞게 완성한다

기본필수	핵심 개념·동작	제공 골격의 지식 CRUD를 끝내고 A/B/C 시나리오를 수행한다. /docs·pgAdmin 수동 확인과 기존 행 보존을 설명한다.
응용	작은 기능 확장	공통 결과 뒤 category 필터 한 개와 함수 분리를 보완한다. 일치 결과·빈 배열·잘못된 입력을 구분한다.
심화	역할 분리·검증	역할별 구조와 전용 테스트 DB의 검증을 보완한다. 트랜잭션·오류 변환·DB 반영을 근거로 설명한다.

공통 완료가 먼저다. 자동 테스트·복잡한 폴더·추가 기능이 기본필수 만점 조건은 아니다.

기본필수만으로 공통 평가 100점

항목	배점	충족 기준
설정·대상 확인	15	Secret 분리 · 실제 연결 DB 확인
API CRUD	25	목록·상세·생성·PUT·삭제와 HTTP 계약
API/DB 모델 연결	15	category 문자열 ⇄ FK · 입력/응답/ORM 구분
오류·트랜잭션	15	404·422 보호 · 제공 rollback 시연 설명
Migration 이해·확인	10	별도 DB에서 제공 변경 적용·실제 열 확인
통합 검증·설명	20	같은 ID DB 대조 · 재실행 · N+2 · 실행 안내

자동 테스트 작성·새 Revision 직접 설계·성능 수치·연속 ID는 공통 만점 조건이 아니다.

세 질문으로 요청 한 건의 흐름을 설명한다

Q1. API가 201을 반환하기 전에 어떤 작업이 성공해야 하는가?

Q2. Pydantic의 category 문자열을 DB에 그대로 넣는가?

Q3. ORM Model을 바꾸면 기존 DB 열도 자동 변경되는가?

1. 지식·분류의 저장 작업과 COMMIT이 성공해야 한다.
2. 아니다. 분류명을 조회·생성하여 category_id로 저장한다.
3. 아니다. 변경 이력을 검토하고 대상 DB에 Migration을 적용한다.

실제 HTTP 응답과 같은 ID의 DB 행을 함께 보여주며 답한다.

오늘의 완료와 다음 단계의 경계를 남긴다

완료: API 요청이 PostgreSQL의 같은 ID 행으로 연결된다.

검증: CRUD·오류·DB 반영·재실행 결과를 근거로 설명한다.

한계: 로컬 학습용이다. 인증·권한·운영 장애·검색 인덱스는 별도다.

다음: 11/13 LLM 기초 → 임베딩·문서 처리 → 11/20 Basic RAG

FastAPI·DB는 AI 플랫폼의 기반이다. 오늘 Agent 자체를 완성한 것은 아니다.

구조는 이해한 책임만큼 나눈다

기본필수 · 평평한 시작

main.py : Schema·Route·작은 도우미
settings.py : 환경 설정
db.py : Engine·Session·transaction
models.py : 두 ORM 클래스
.env : 실제 비밀, 제출 제외

응용 → 심화 · 필요한 만큼

응용: schemas.py·조회 함수 분리

심화: app/routers · services
repositories · models · schemas
tests/ · migration_demo/

기능·외부 계약은 동일

기본필수 main:app | 심화 app.main:app | 프로젝트 최상위에서 실행

settings.py · 경로가 고정된 설정 읽기

```
from pathlib import Path
from pydantic import SecretStr
from pydantic_settings import (
    BaseSettings, SettingsConfigDict,
)
class Settings(BaseSettings):
    db_host: str = "localhost"
    db_port: int = 5432
    db_name: str
    db_user: str
    db_password: SecretStr
    model_config = SettingsConfigDict(
        env_file=Path(__file__).resolve().parent / ".env",
        env_file_encoding="utf-8",
    )
settings = Settings()
```

기본필수 경로

settings.py와 같은 폴더의 .env

필수 값 누락은 설정 오류다.
SecretStr은 출력 가림 용도이며
보관 파일 암호화가 아니다.

심화 이동 시 기준 경로도 수정.

실제 비밀번호 출력 금지
DB 대상만 로컬에서 확인

db.py · 연결과 Session의 준비

```
from sqlalchemy import create_engine, URL
from sqlalchemy.orm import sessionmaker
from settings import settings
url = URL.create("postgresql+psycopg",
    username=settings.db_user,
    password=settings.db_password.get_secret_value(),
    host=settings.db_host, port=settings.db_port,
    database=settings.db_name)
engine = create_engine(url, pool_pre_ping=True,
    hide_parameters=True,
    connect_args={"connect_timeout": 5})
SessionLocal = sessionmaker(bind=engine,
    autoflush=False, expire_on_commit=False)
def get_db():
    with SessionLocal() as db:
        yield db
```

함께 있어야 할 코드

본문의 transaction 도우미도
이 모듈에 둔다.

SessionLocal은 Factory다.
요청마다 SessionLocal() 호출

Engine은 요청마다 만들지 않는다.

코드가 정상 import돼도
접속 확인은 별도로 필요

models.py · 공통 Base와 import

```
from sqlalchemy import (
    BigInteger, String, Text, Identity,
    ForeignKey, CheckConstraint,
)
from sqlalchemy.orm import (
    DeclarativeBase, Mapped, mapped_column,
)
class Base(DeclarativeBase):
    pass

# 이어서 본문의 Category 클래스 정의
# 이어서 Knowledge 필드 + A05의 제약 정의
# 기존 DB 연결 앱에서는 create_all을 호출하지 않는다.
```

기본 정의

Base: ORM metadata의 기반
Mapped: Python 속성의 타입
mapped_column: 실제 열 매핑

Model import와 SQL 실행은
같은 일이 아니다.

public.categories
public.knowledge

Knowledge의 길이 CHECK와 스키마를 빠뜨리지 않는다

```
# Knowledge 클래스 내부 · 필드 선언 위에 포함
__table_args__ = (
    CheckConstraint(
        "char_length(title) BETWEEN 1 AND 80",
        name="knowledge_title_length",
    ),
    CheckConstraint(
        "char_length(content) BETWEEN 1 AND 2000",
        name="knowledge_content_length",
    ),
    {"schema": "public"},
)
```

실제 DB와 일치

NOT NULL과 CHECK는 다르다.

제목·본문 빈 문자열을 막는다.
공백만인 값은 현재 계약에 포함된다.

제목 UNIQUE는 추가하지 않는다.

선언 누락이 있어도
기존 DB 제약은 별도로 남을 수 있음

main.py · 연결된 예제의 선행 정의

```
from typing import Annotated
from fastapi import (
    FastAPI, Depends, HTTPException, Response,
)
from pydantic import BaseModel, Field
from sqlalchemy import select
from sqlalchemy.orm import Session
from db import get_db, transaction
from models import Knowledge, Category

app = FastAPI(title="Knowledge API · PostgreSQL")
DbSession = Annotated[Session, Depends(get_db)]
@app.get("/health")
def health():
    return {"status": "ok"}
```

이어서 놓을 코드

KnowledgeWrite·Read
to_read / 분류 도우미 / 404 도우미
GET·POST·PUT·DELETE
예외 처리기

같은 경로는 한 번만 등록한다.

/health는 앱 응답 점검
DB 전체 준비 상태 보장은 아님

category 필터 · SQL 조건으로 연결한다

```
@app.get("/api/knowledge",
         response_model=list[KnowledgeRead])
def list_knowledge(db: DbSession,
                   category: str | None = None):
    stmt = select(Knowledge, Category.name).join(
        Category, Knowledge.category_id == Category.id)
    if category is not None:
        stmt = stmt.where(Category.name == category)
    stmt = stmt.order_by(Knowledge.id)
    return [to_read(row, name)
            for row, name in db.execute(stmt).all()]
```

기존 목록 함수를 교체

Query 생략 → 전체 목록
일치 분류 → 해당 지식
결과 없음 → 200 · []

비교값은 바인딩된다.
SQL 문자열 덧붙이기는 피한다.

문자열 분류 필터
벡터 검색·RAG가 아님

기존 계약을 몰래 바꾸지 않는다

입력·상황	이번 계약	주의
같은 title로 다시 생성	허용 · 새 id의 지식	title UNIQUE 없음
새 category 문자열	길이가 유효하면 분류를 조회·생성	분류와 지식을 한 번에 commit
PUT에 content만	422	세 필드가 모두 필수
추가 입력 필드	기본 Pydantic 정책: 무시	extra forbid를 임의 추가하지 않음
공백만인 문자열	길이 규칙상 허용	trim·업무 정책은 별도 합의
DELETE 후 부모 분류	그대로 유지	빈 분류 삭제는 별도 기능

HTTP 변환은 알려진 오류만 정확히 처리한다

```
from fastapi import Request
from fastapi.responses import JSONResponse
from sqlalchemy.exc import IntegrityError
@app.exception_handler(IntegrityError)
async def integrity_error(request: Request, exc):
    code = getattr(exc.orig, "sqlstate", None)
    diag = getattr(exc.orig, "diag", None)
    name = getattr(diag, "constraint_name", None)
    known = {("23505", "categories_name_key"),
             ("23503", "knowledge_category_fk")}
    status = 409 if (code, name) in known else 500
    detail = ("Data conflict" if status == 409 else
              "Database operation failed")
    return JSONResponse(status_code=status,
                        content={"detail": detail})
```

쓰기 취소 뒤 변환

transaction에서 rollback
→ 예외 재전달
→ 이 처리기가 HTTP로 변환

원문 SQL·파라미터·진단 문자열을
그대로 응답하지 않는다.

같은 IntegrityError라도
모두 중복·409는 아니다.

접속 오류도 설정·운영 오류와 구분한다

```
from sqlalchemy.exc import DBAPIError, OperationalError
@app.exception_handler(DBAPIError)
async def other_db_error(request: Request, exc):
    code = getattr(exc.orig, "sqlstate", None)
    unavailable = (
        isinstance(exc, OperationalError)
        and (str(code or "").startswith("08")
             or bool(exc.connection_invalidated))
    )
    status = 503 if unavailable else 500
    detail = ("Database unavailable" if unavailable
              else "Database operation failed")
    return JSONResponse(status_code=status,
                        content={"detail": detail})
```

수업의 제한된 정책

연결 계열 SQLSTATE 08* 또는
확인된 연결 무효화를 503 후보로 둔다.

인증·잘못된 SQL·예상 밖 오류는
같은 것으로 처리하지 않는다.
원인은 로컬에서 안전하게 점검.

성공처럼 200을 반환하지 않음
실제 배포 전 환경별 검증 필요

같은 Session에서 실패 후 rollback을 확인한다

```
# day19_api_test 전용 · 기존 learning 분류 전제
with TestSessionLocal() as db:
    try:
        db.add(Category(name="learning"))
        db.flush()
    except IntegrityError as exc:
        print(exc.orig.sqlstate)  # 기대: 23505
        db.rollback()
    else:
        db.rollback()
        raise AssertionError("중복 실패가 필요합니다.")
    # 같은 db 객체로 후속 조회
    count = db.scalar(select(func.count(Category.id)))
    print("category count:", count)
```

관찰해야 할 것

실패한 DB 제약이 UNIQUE인가?
rollback 줄을 실행했는가?
같은 db로 조회가 가능한가?
기존 행은 그대로인가?

func는 sqlalchemy에서 import.

새 요청이 성공한 것과
동일 Session 회복은 다르다.

연습 DB 전용 Engine을 Alembic에 전달한다

```
# migration_demo/migrations/env.py의 online 핵심
# demo_url: .env.migration으로 만든 URL 객체
connectable = create_engine(demo_url,
                            poolclass=pool.NullPool)
with connectable.connect() as connection:
    actual = connection.execute(text(
        "SELECT current_database()"
    )).scalar_one()
    connection.rollback()  # 점검 SELECT의 tx 종료
    if actual != "day19_migration_lab":
        raise RuntimeError("연습 DB 대상 확인 필요")
    context.configure(connection=connection,
                      target_metadata=target_metadata)
    with context.begin_transaction():
        context.run_migrations()
```

중요한 안전장치

Model을 import해 metadata 등록
.env.migration 경로 고정
실제 DB명 검사

URL 객체를 직접 전달하므로
ini의 % 보간과 분리한다.

기존 .env를 우연히 읽어
주 DB를 변경하지 않도록 한다.

Migration 예제의 처음·중간·끝 상태

상태	표·열·데이터	이력 확인
시작	새 day19_migration_lab · 사용자 테이블 없음	alembic current: 적용 없음
001_initial 적용	categories·knowledge 생성 · 기존 DDL과 동일	현재 001_initial
기준 데이터	가상 분류와 지식 1건 COMMIT	id·원래 열 값 기록
002_source 적용	source_label VARCHAR(120) NULL 추가	현재 002_source · head
되돌림 시연(선택)	새 열 삭제 · 그 열 값은 복구되지 않음	001_initial로 돌아감

001의 upgrade는 부모→자식 생성, downgrade는 자식→부모 삭제한다. 삭제 시연은 연습 DB만 사용.

참고 S12 · S14 | 공식 링크: 발표자 노트·참고자료

테스트는 요청마다 Test Session을 주입한다

```
# TestSessionLocal은 전용 PostgreSQL DB에 연결
from fastapi.testclient import TestClient

def override_get_db():
    with TestSessionLocal() as db:
        yield db

app.dependency_overrides[get_db] = override_get_db
try:
    with TestClient(app) as client:
        # 이 안에서 API 요청과 Test DB 행을 대조
        run_contract_checks(client, TestSessionLocal)
finally:
    app.dependency_overrides.pop(get_db, None)
```

준비 코드는 별도

실제 DB 대상 검사
테이블·기준 데이터 준비
테스트가 만든 행만 정리

같은 이름의 함수라도
앱이 쓰는 get_db 객체를 교체한다.

TestClient ≠ 실서버 재시작
mock 통과 ≠ PostgreSQL 통과

정상·경계·실패 확인표

대상	예상 HTTP	추가 확인
기존/새 분류로 등록	201	새 분류 재사용·동일 id DB 행
제목 중복	201	새 id 생성 · 원래 지식 보존
없는 ID + 정상 PUT	404	새 category도 생성하지 않음
길이 초과 / null / 누락	422	지식·분류 기존 내용 불변
알려진 DB 제약 충돌	409	rollback · 내부 정보 미노출
예상 밖 CHECK 실패	500	성공으로 위장하지 않음
삭제 · 재실행	204 / 이후 GET404	다른 행·B 수정 내용 유지

실행 위치·가상환경·오전 서버를 구분한다

```
# 활성화 정책이 막힌 경우: 정책 우회 대신 직접 실행
.\.venv\Scripts\python.exe -m pip show sqlalchemy
.\.venv\Scripts\python.exe -m uvicorn main:app --port 8001

# 심화 app/ 구조라면 프로젝트 최상위에서
python -m uvicorn app.main:app --port 8001 --reload

# 브라우저 주소
http://127.0.0.1:8001/docs

# PostgreSQL 포트와 API 포트는 다른 목적이다.
# DB: 설치 포트(예:5432) / API: 8001
```

둘 중 하나만 실행

main:app → 루트 main.py
app.main:app → app/main.py

명령은 터미널에 입력한다.
URL은 브라우저에서 연다.

기존 서버는 Ctrl+C로 종료.

Windows 직접 실행은
제작 환경에서 미검증

공식 참고자료 · API·설정·ORM

ID	공식 자료	확인한 내용
S01	FastAPI · Dependencies with yield	요청별 준비·정리
S02	SQLAlchemy · Session Basics	flush·commit·rollback·close
S03	SQLAlchemy · ORM Quick Start	클래스·열·조회·변경
S04	SQLAlchemy · Engine / URL.create	드라이버·연결·특수문자
S05·S06	FastAPI Settings / Psycopg Install	환경변수·.env·설치명
S07~S09	FastAPI Errors / Response / Status	오류·응답 모델·204

공식 링크는 각 슬라이드 발표자 노트에 기록했다. 확인일: 2026-11-08

참고 S01 · S02 · S03 · S04 · S05 · S06 · S07 · S08 · S09 | 공식 링크: 발표자 노트·참고자료

공식 참고자료 · DB·Migration·검증

ID	공식 자료	확인한 내용
S10·S11	PostgreSQL 18 · Constraints / Sequence	무결성·ID 빈 번호
S12~S14	Alembic · Tutorial / Autogenerate / Operations	이력·초안 검토·열 변경
S15	FastAPI · Testing Dependencies	Test Session 교체·원복
S16·S17	SQLAlchemy Exceptions / Psycopg Errors	예외 종류·진단값
S18	FastAPI · async / await	동기 경로·드라이버 구분
S19·S20	PostgreSQL 18 · Transactions / Error Codes	원자성·SQLSTATE

기존 상품·주문 예제 대신 오전 Spec_v2의 지식 API·두 테이블 계약을 사용한다.

참고 S10 · S11 · S12 · S13 · S14 · S15 · S16 · S17 · S18 · S19 · S20 | 공식 링크: 발표자 노트·참고자료

검수한 것과 아직 확인하지 못한 것을 구분한다

범위	이번 제작에서 수행	판정
PPT	전 슬라이드 렌더링·내용·배치 점검	최종 산출물 기준 재확인
코드·metadata	Python 문법·PostgreSQL DDL·Revision SQL 생성	실행 DB 적용과는 다름
모의 API/Session	입력·응답·정렬·오류 매핑·취소 호출 69항목	통과 · 모의 객체 검수
실제 PostgreSQL	접속·CRUD·제약·Migration·재접속	미검증
Windows·pgAdmin	설치·UI·실제 버튼 조작	미검증

교실 사전 확인: 실제 DB 대상·구조 → API CRUD → 실패 rollback → 별도 Migration·Test DB